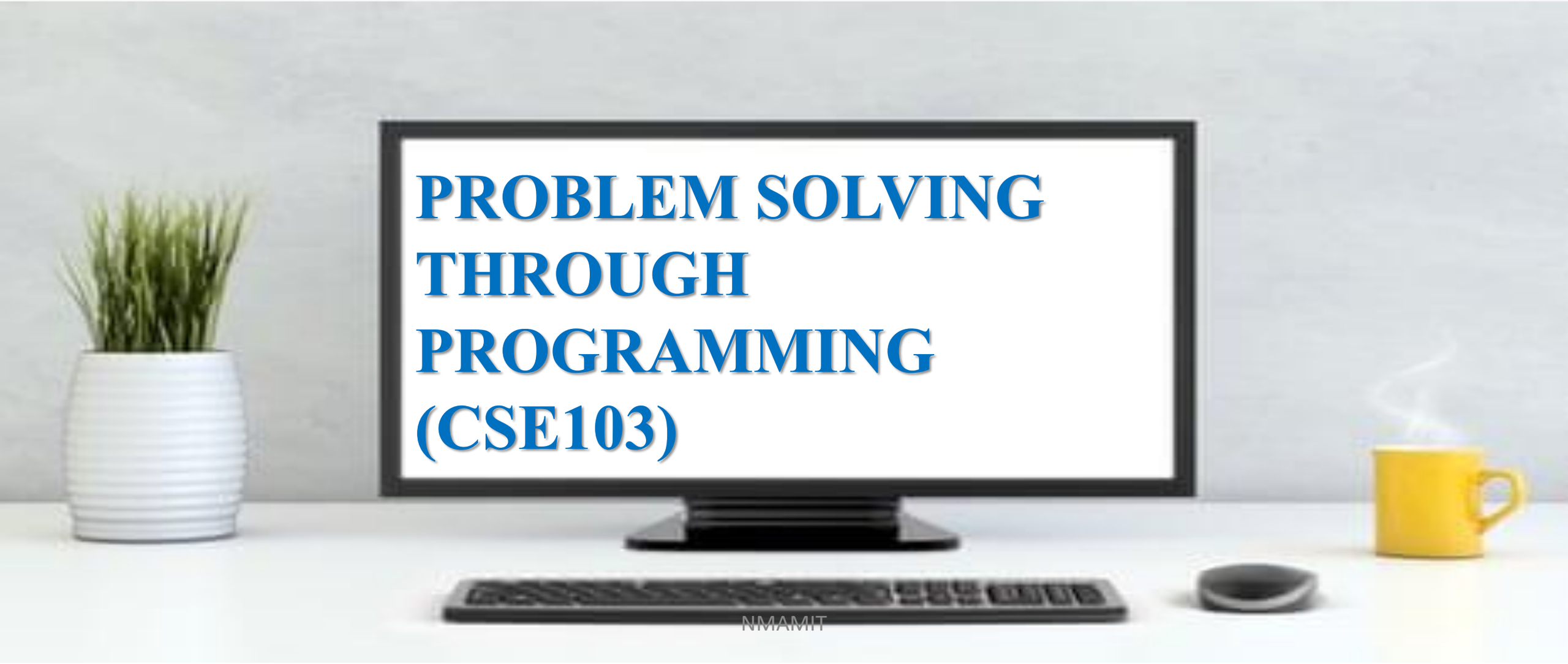
NITTE
(Deemed to be University)

NMAM INSTITUTE OF TECHNOLOGY



PROBLEM SOLVING THROUGH PROGRAMMING (CSE103)

Unit 2

C Programming

Syllabus:

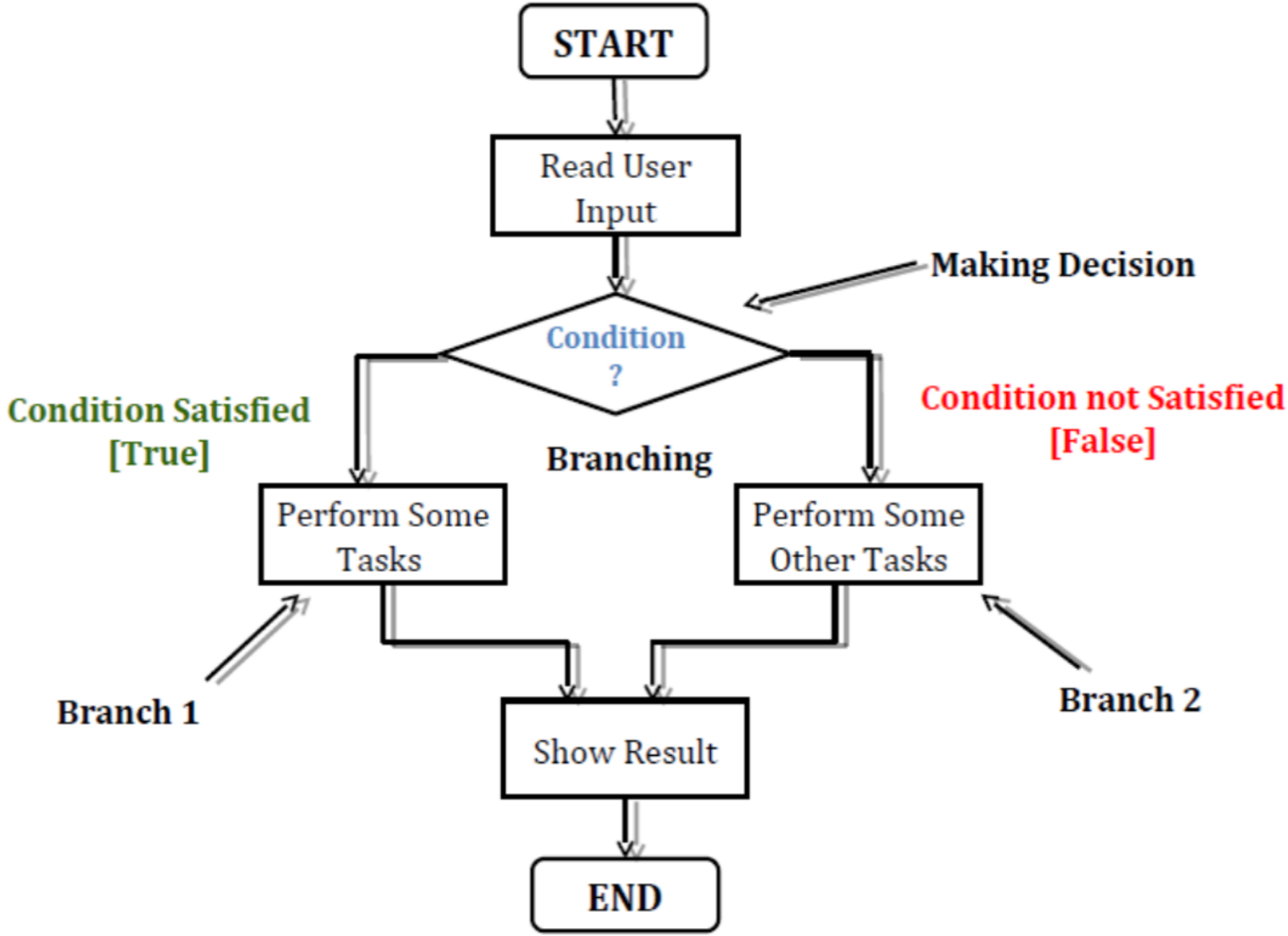
Decision making and Branching: Decision making with if statement, Simple if Statement, the if...else statement, Nesting of if...else statements, The else...if ladder, The switch statement.

Decision making and Looping: The for statement, the while statement, the do...while statement, Jump statements (goto, break, continue).

Decision making an Branching

INTRODUCTION:

- Generally C program execute it's statements sequentially. But in order to solve problems we may have some situations where we have to change the order of executing the statements based on whether some conditions have met or not.
- So controlling the execution of statements based on certain condition or decision is called decision making and branching.


START
Read User Input
Making Decision
Condition ?
Condition Satisfied [True]
Condition not Satisfied [False]
Branching
Perform Some Tasks
Perform Some Other Tasks
Branch 1
Branch 2
Show Result
END

Types of Control Statements are:

(1) Conditional Branching Control Statements

(a) **if** statement

(b) **if-else** statement

(c) **nested if-else** statement

(d) **else if** ladder

(e) **switch** statement

(2) Unconditional Branching Control Statement

(a) **goto** statement

Simple if:

It is called one way branching statement.

It involves a test expression (a relational or conditional expression).

```
if (condition)
{
statement; // body of if
}
statement –x;
```

Syntax- if (Condition)

{

statement block;

}

statement x ;

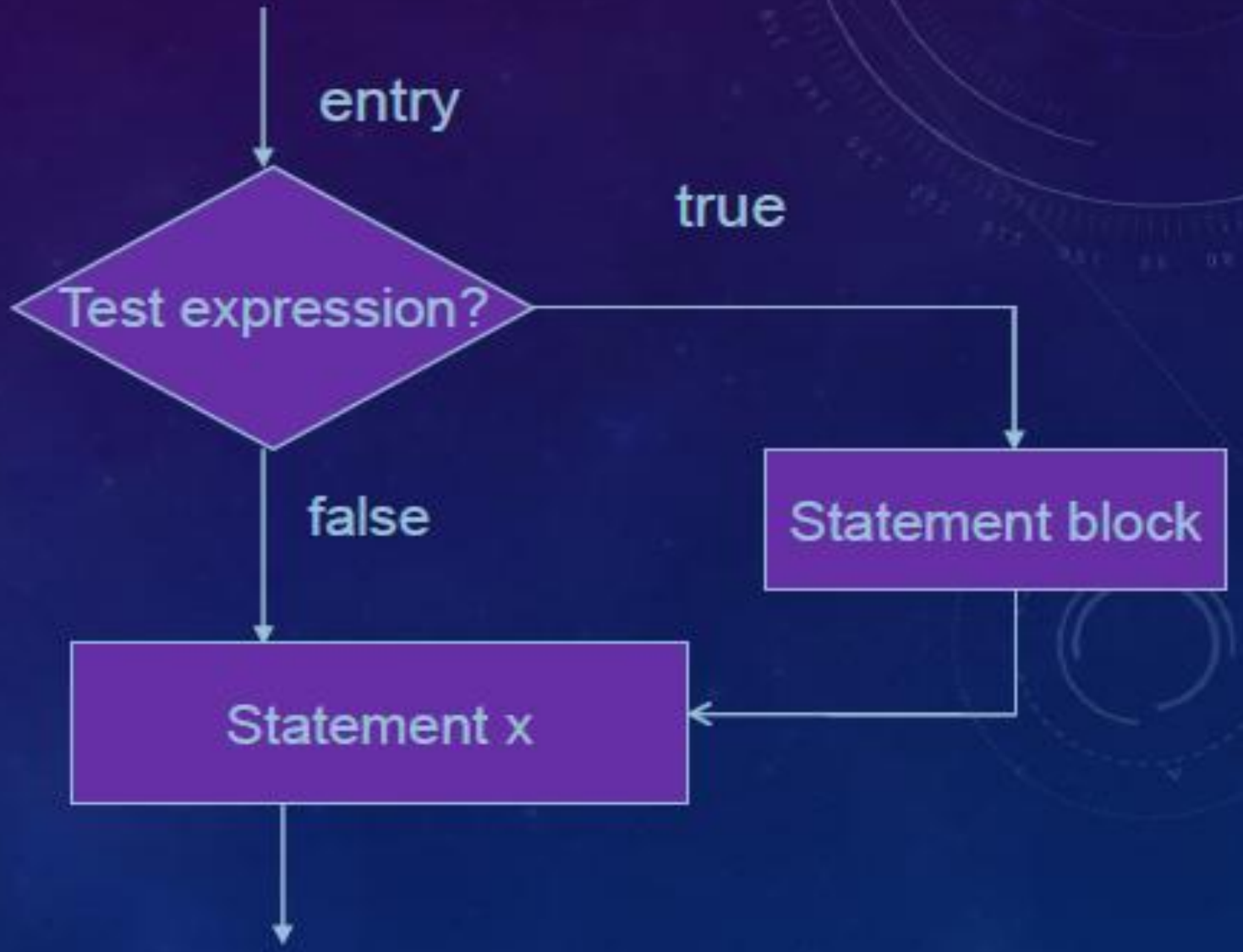
Ex: if (category == sports)

{

marks=marks+bonus;

}

printf (“%f”,marks);



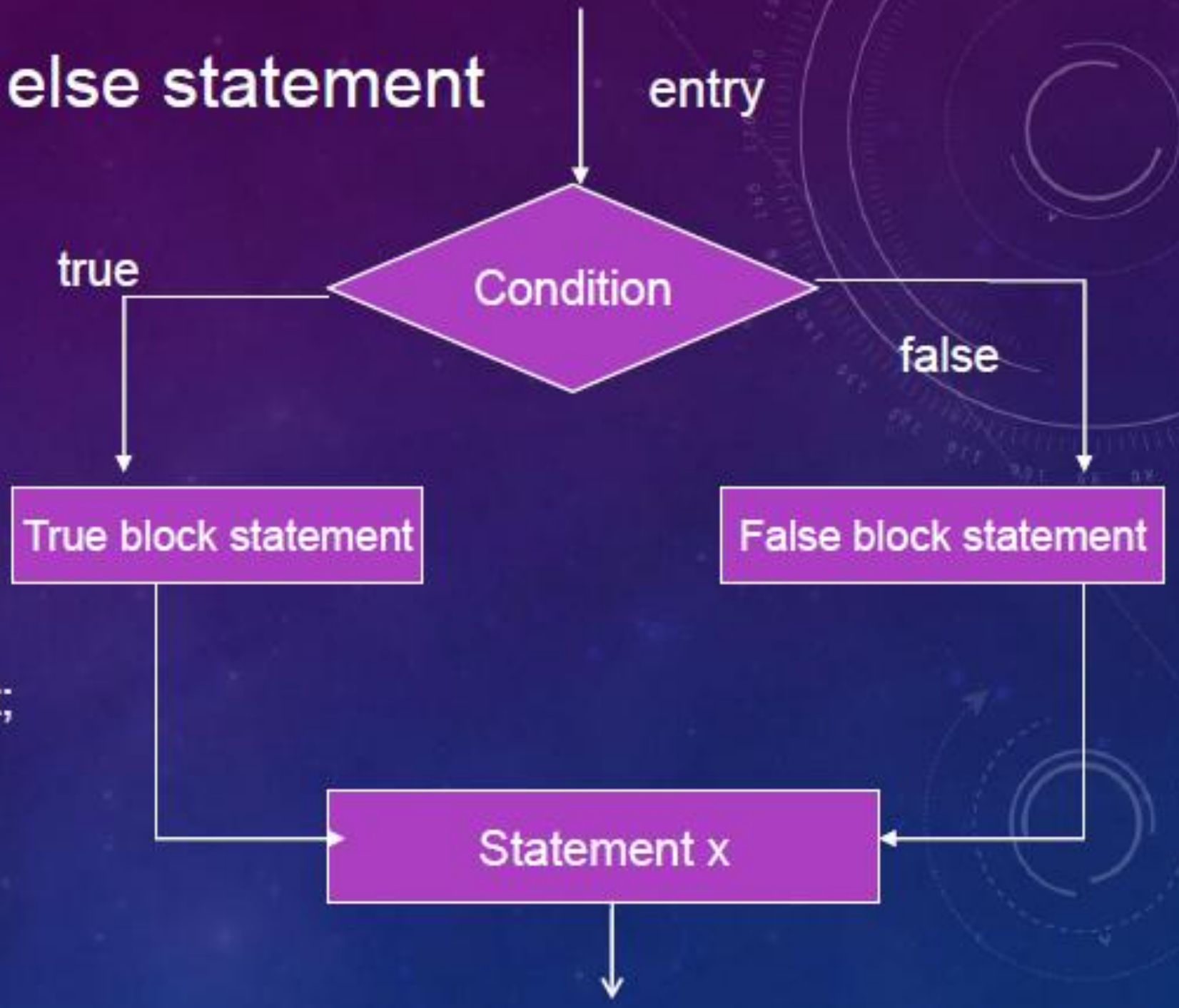
```
int number=15
if(number%2==0)
{
printf(“This is even number\n");
}
printf(“The number is %d\n“,number);
```

Else if statement

- The **if** statement provides one way branching, i.e., it executes the statement/s associated with **if**, only when the test expression is true, but does nothing if it is false.
- The **if-else** statement is an extension of the **if** statement. The **if-else** statement is a *two way*
- *branching*, i.e., it executes one set of statement/s if the test expression is true or it executes another set of statement/s if the test expression is false.

The if…else statement

```
Syntax-   if( Condition)
   {
      true block statement;
   }
 else
  {
      false block statement;
   }
 statement x;
```



Program for if…else statement

```
main()
{
 int a;
 printf("Enter an integer");
 scanf("%d",&a);
  if(a%2==0)
  printf(" %d is even number",a);
     else
   printf("%d is an odd number",a);
}
```

output- Enter an integer

46

46 is an even number

NESTED IF STATEMENTS

When a series of decisions are involved, we may have to use more than one **if-else** statement. Enclosing one **if** within another is called **nested if** statement.

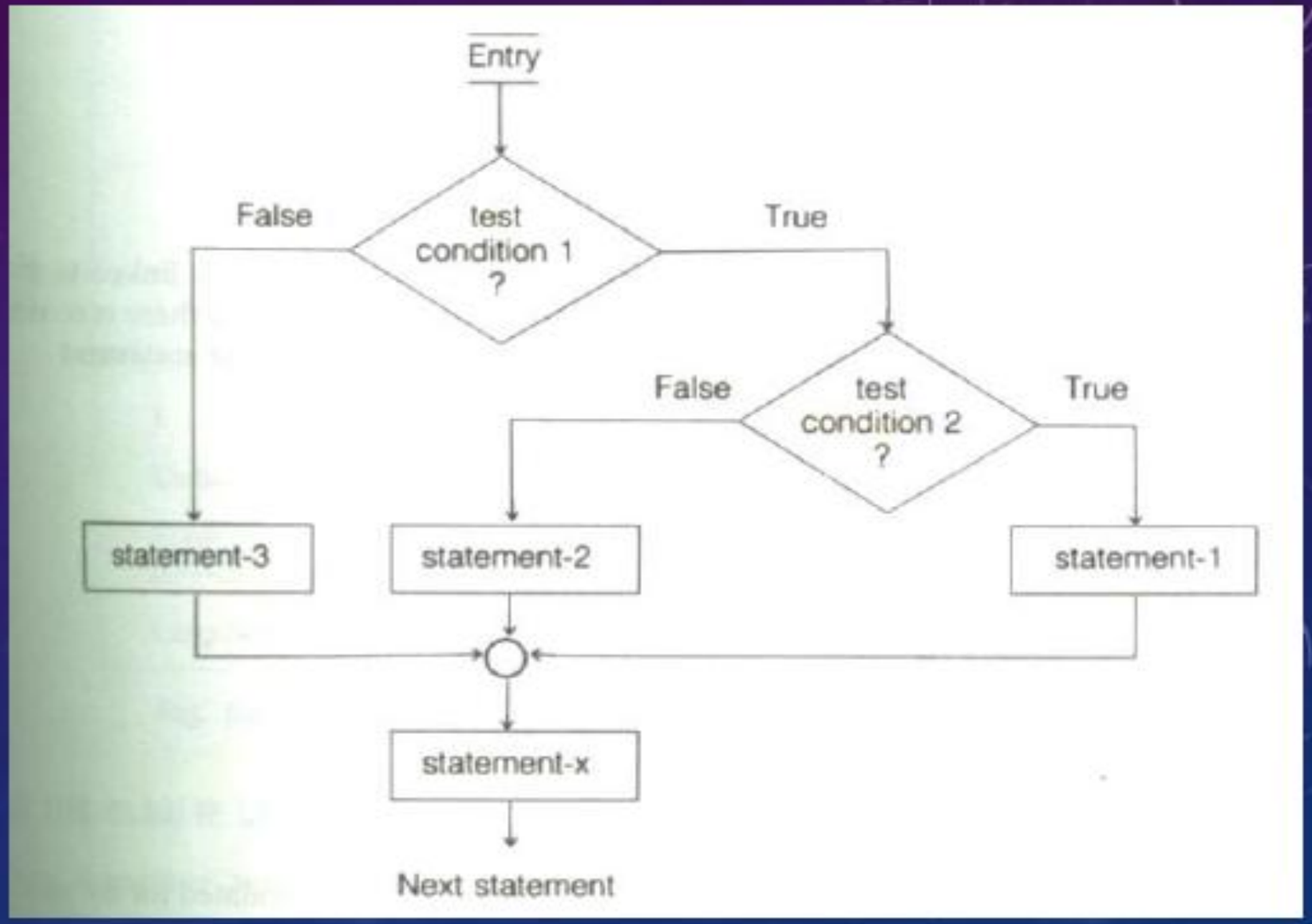
Syntax:

```
if (condition1)
{
if (condition2)
{
statement1;
}
else
{
Statement 2
}
}
```

NESTING OF IF...ELSE STATEMENT

Syntax-

```
if( test condition 1)
    {
    if (test condition 2)
      {
      statement 1;
      }
    else
      {
      statement 2;
      }
    }
else
   {
    statement 3;
   }
statement x  ;
```



Largest of three numbers a,b,c:

```
if(a>b){
        if(a>c){
                 printf("\n%d is greatest number ",a);
                }else{
                        printf("\n%d is greatest number ",c);
                       }
          }else if(b>a){
                        if(b>c){
                                        printf("\n%d is greatest number ",b);
                         }else{
                                         printf("\n%d is greatest number ",c);
}
```

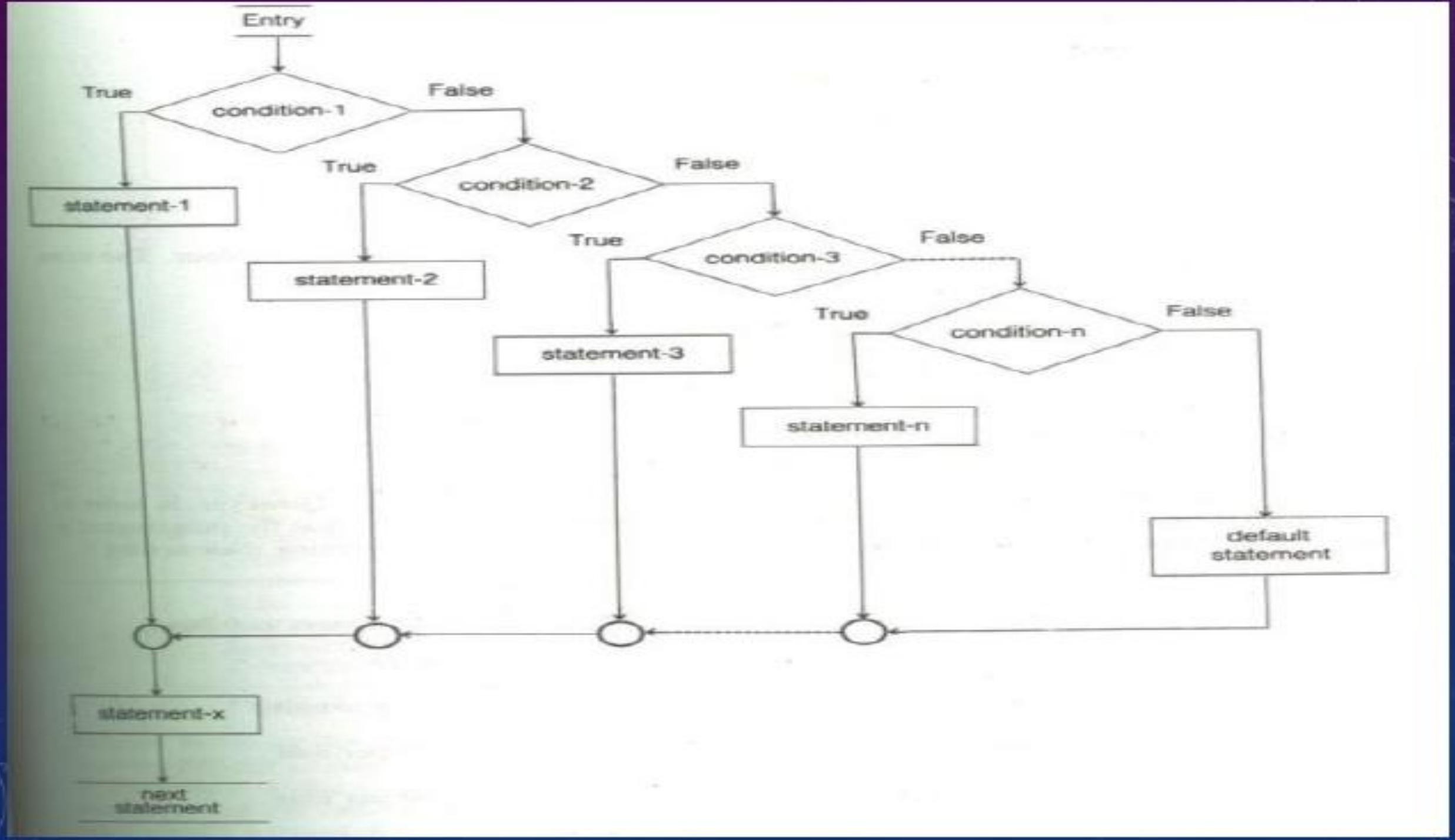
THE ELSE-IF LADDER

When multipath decisions are involved, we can have a chain of **if**s in which the statements associated with each **else** is an **if**.

The else if ladder

syntax-:

```
if  ( condition 1)
  statement 1 ;
    else if ( condition 2)
      statement 2 ;
        else if ( condition 3)
          statement 3 ;
            else if( condition n)
              statement n ;
                else
                  default statement ;
      statement x;
```


Entry
True
condition-1
False
statement-1
True
condition-2
False
statement-2
True
condition-3
False
statement-3
True
condition-n
False
statement-n
default
statement
statement-x
next
statement

SWITCH STATEMENT

The **nested if** allows selecting one of the many alternatives but it is time consuming because it tests all the conditions and based on the result a particular branch is taken for execution.

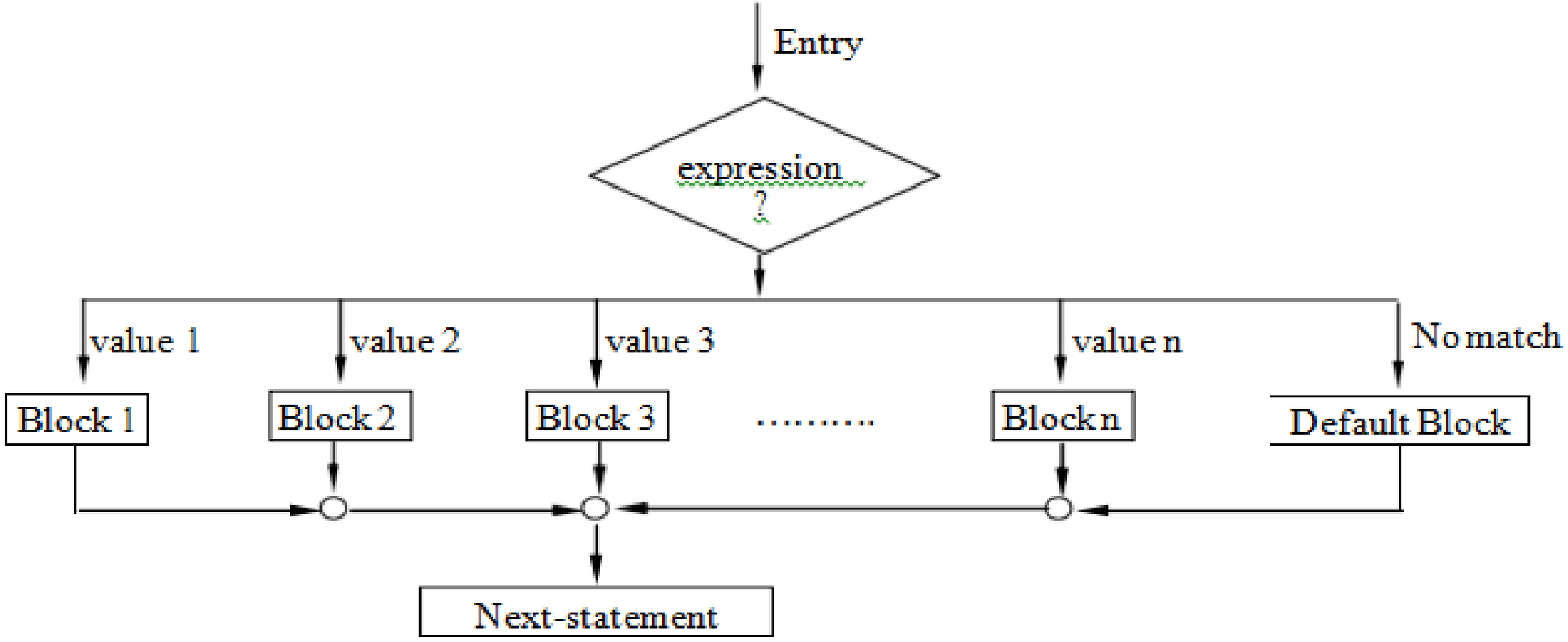
To overcome this disadvantage, the **switch** statement is used. The **switch** statement provides a ***multiple way branching***.

Syntax:

```
switch (expression)
{
case value1: block1;
break;
case value2: block2;
break;
case value3: block3;
break;
…….
case value n:block n;
break;
default: default block;
break;
}
```

- The use of **break** is very important here. The block of statements corresponding to each case ends with a **break** statement. What if the **break** statement is not used?
- The switch-case works like this: As the program enters the switch construct, it starts comparing the value of switching expression with the cases, and executes the block of its first match. The **break** causes the control to go out of the switch scope. If **break** is not found, the subsequent block also gets executed, leading to incorrect result.

Flowchart:


Entry
expression
?
value 1
value 2
value 3
value n
No match
Block 1
Block 2
Block 3
.....
Block n
Default Block
Next-statement

```
int main() {
    int day;
    printf("Enter day number (1 to 7): ");
    scanf("%d", &day);
    switch(day) {
        case 1: printf("Sunday\n"); break;
        case 2: printf("Monday\n"); break;
        case 3: printf("Tuesday\n"); break;
        case 4: printf("Wednesday\n"); break;
        case 5: printf("Thursday\n"); break;
        case 6: printf("Friday\n"); break;
        case 7: printf("Saturday\n"); break;
        default: printf("Invalid day number\n");
    }
    return 0;
}
```

Write a C program to implement a simple calculator.

```
int main ()
{
int a = 10, b = 5;
int op = 1;
float result;
printf("1: addition\n 2: subtraction\n 3: multiplication\n 4: division\n ");
printf("\na: %d b: %d : op: %d\n", a, b, op);
switch (op){
case 1: result = a + b; break;
case 2: result = a - b; break;
case 3: result = a * b; break;
case 4: result = a / b; break;
default: printf("Invalid operation\n");
}
printf("Result: %6.2f", result); return 0; }
```

Example 2

```
main()
{
 int grade,mark,index;
 printf("Enter ur marks \n");
 scanf("%d",&mark);
 index=mark/10;
 switch(index)
 {
  case 10:
 case 9:
 case 8:
     grade="Honours";
     break;
 case 7:
 case 6:
     grade="First Division";
     break;
 case 5:
     grade="Second Division";
     break;

  case 4:
     grade="First Division";
     break;
   default:
      grade="Fail";
      break;
    }
   printf("%s",grade);
}
```

Decision making and Looping in C Language

Contents:

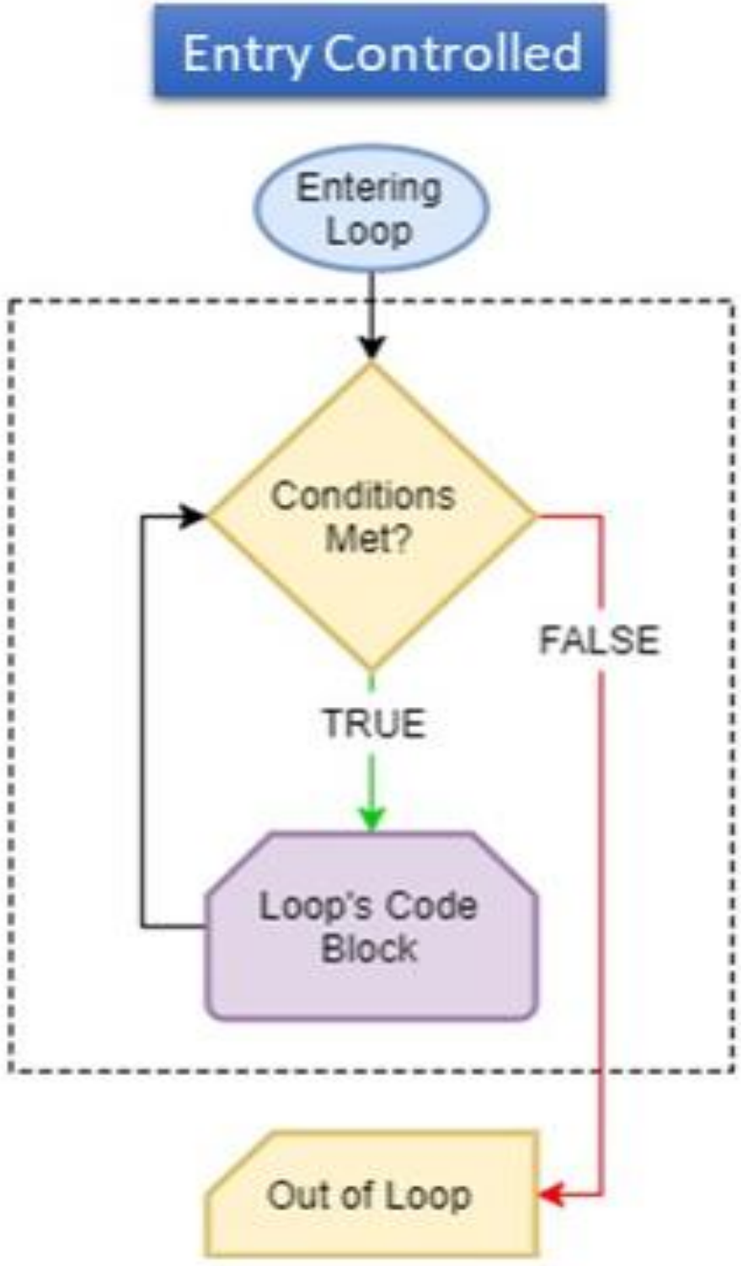
- while statement
- do...while statement
- for statement
- Jumps in Loops.
- break and continue statements.
- goto statement
- return statement

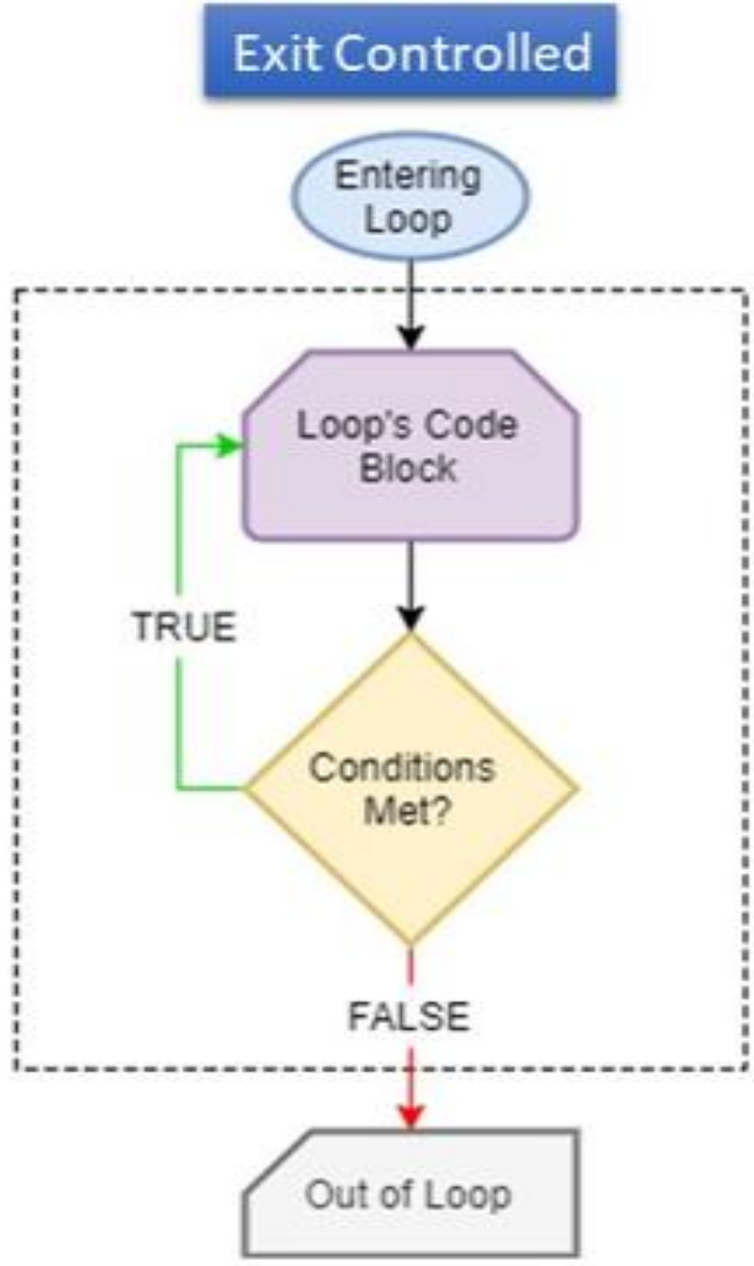
Looping

- Loops in programming are used to repeat a block of code until the specified condition is met.
- A loop statement allows programmers to execute a statement or group of statements multiple times without repetition of code.

- **There are mainly two types of loops in C Programming:**

1. **Entry Controlled loops:** In Entry controlled loops the test condition is checked before entering the main body of the loop. **Ex: For Loop and While Loop are Entry-controlled loops.**
2. **Exit Controlled loops:** In Exit controlled loops the test condition is evaluated at the end of the loop body. The loop body will execute at least once, irrespective of whether the condition is true or false. **Ex: do-while Loop is Exit Controlled loop.**


Entry Controlled
Entering Loop
Conditions Met?
FALSE
TRUE
Loop's Code Block
Out of Loop


Exit Controlled
Entering Loop
Loop's Code Block
TRUE
Conditions Met?
FALSE
Out of Loop

while loop

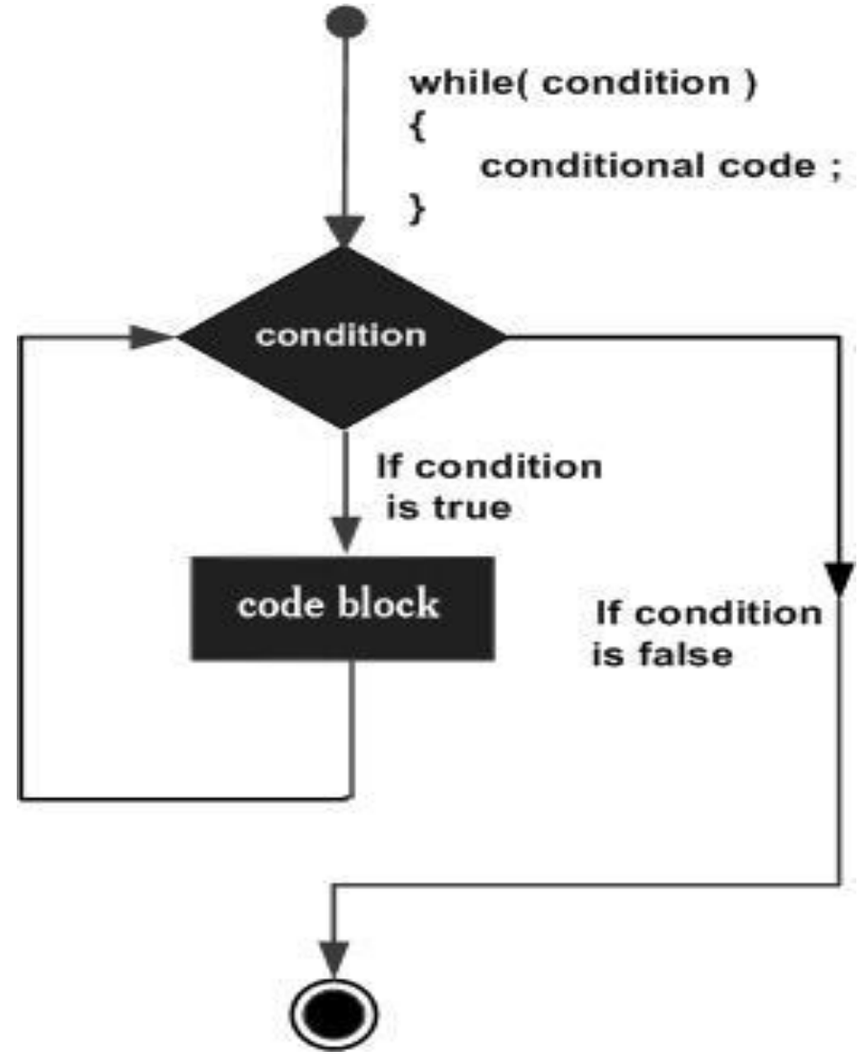
- It executes a set of statements repeatedly as long as the specified condition is true.
- Entry-controlled loop.

Syntax of while statement:

```
while (test_condition)
{
        statements;
}
statement x;
statement y;
```

while is a keyword.
test_condition is a logical or relational expression that results in either TRUE or FALSE.

```
while (test_condition)
{
    statement1;          // compound statement
    statement2;
    statement3;
}
 Statement x;
 Statement y;
```



Example

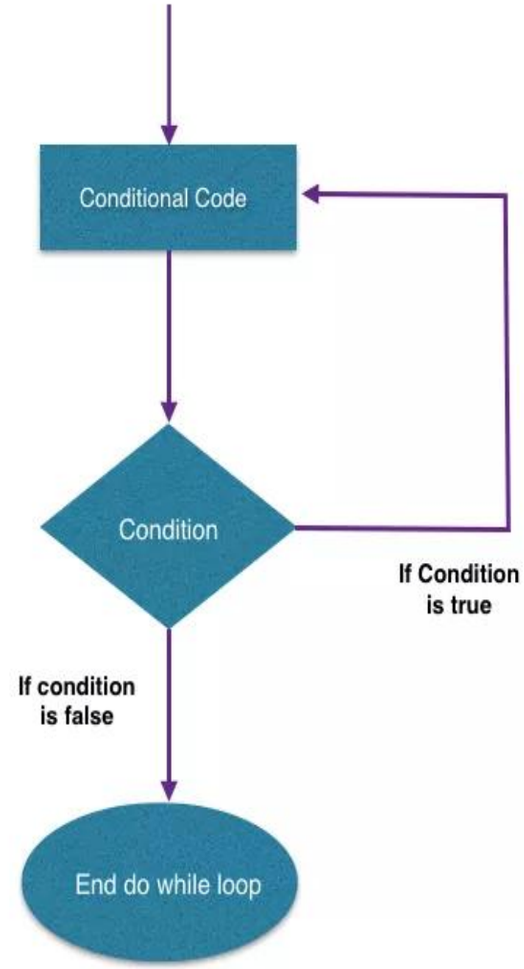
```
#include<stdio.h>
int main ()
{
        int a = 10;
        while( a < 20 )
        {
                printf("value of a: %d\n", a);
                a++;
        }
 return 0;
}
```

```
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
value of a: 16
value of a: 17
value of a: 18
value of a: 19
```

do..while statement

Syntax of do..while statement:

```
do
{
        statement1;
         statement2;
} while (test_condition);
Statement x;
Statement y;
```



- do and while are keywords.
- test_condition is a logical or relational expression that results in either TRUE or FALSE.

- On reaching the **do** statement, the program proceeds to evaluate the body of the loop first. At the end of the loop, the test_condition in the **while** statement is evaluated.
- If the condition is *true*, the program continues to evaluate the body of the *loop* once again. This process continues as long as the *condition* is true.
- When the condition becomes *false*, the *loop* will be terminated and the control goes to the statement that appears immediately after the while statement.
- Since the test condition is evaluated at the bottom of the loop, the

Example

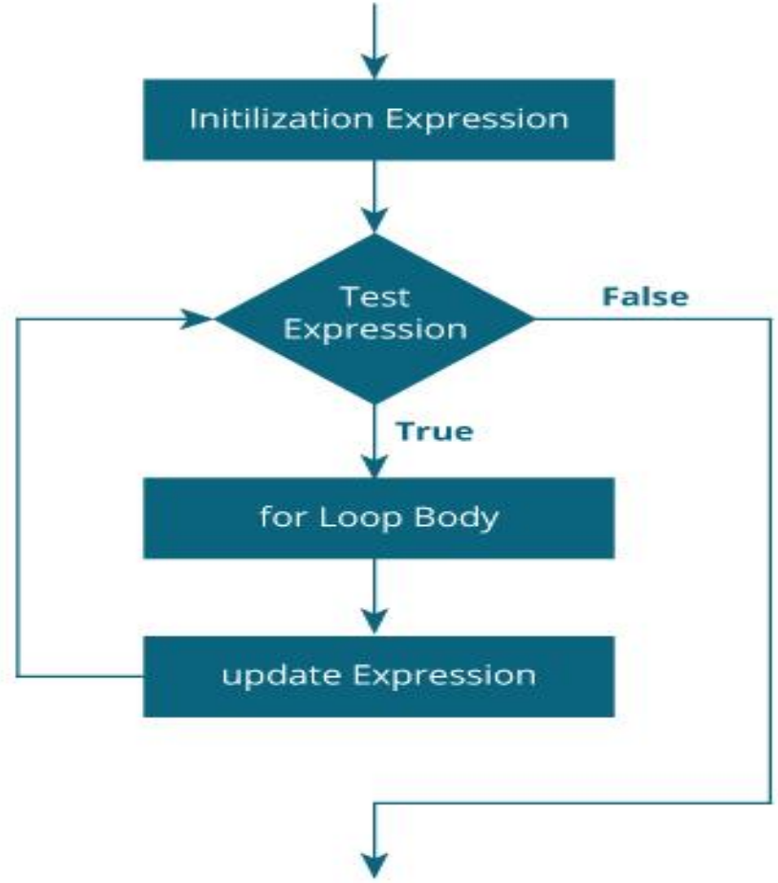
```
#include<stdio.h>
int main ()
{
        int a = 10;
        do
        {
                printf("value of a: %d\n", a);
                a++;
        } while( a < 20 ) ;
 return 0;
}
```

OUTPUT

```
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
value of a: 16
value of a: 17
value of a: 18
value of a: 19
```

for statement

- for is an *entry – controlled loop* statement.
- This statement is used when the programmer knows how many times a set of statements are executed.



Syntax of for statement:

```
for (initialization; test_condition; update_expressio
{
        statement1;
        statement2;
}
Statement x;
```

- **Initialisation** of the *control variable* is done first. Using assignment statements such as i = 1 and count = 0. The variables *i* and *count* are known as *loop – control variables*.
- The value of the *control variable* is tested using the **test_condition**. If the condition is *true*, the body of the loop is executed; otherwise the loop is terminated and the execution continues with the statement the immediately follows the loop.
- When the body of the loop is executed, the control is transferred back to the **for** statement after evaluating the last statement in the loop. Now, the *control variable* is **incremented** using an assignment statement such as i = i+1 and the new value of the *control variable* is again tested to see whether it satisfies the loop condition.

```c
#include <stdio.h>

int main ()
{
    int a;
    for(a=10;a<20;a++)
    {
        printf("value of a: %d\n", a);
    }
 return 0;
}
```

//Factorial of a given number using while loop

```c
#include <stdio.h>
int main()
{
    int n,i,f;
    f=i=1;
    printf("Enter a Number to Find Factorial: ");
    scanf("%d",&n);
    while(i<=n)
    {
        f*=i;
        i++;
    }
    printf("The Factorial of %d is : %d",n,f);
    return 0;
}
```

```c
//Program to print table for the given number using do while loop

#include<stdio.h>
int main(){
int i=1,number=0;
printf("Enter a number: ");
scanf("%d",&number);
do
{
    printf("%d \n",(number*i));
    i++;
}while(i<=10);
return 0;
}
```

```
enter number5
5
10
15
20
25
30
35
40
45
50
```

```c
//Multiplication Table Up to 10
#include <stdio.h>
int main() {
  int n, i;
  printf("Enter an integer: ");
  scanf("%d", &n);
  for (i = 1; i <= 10; ++i) {
    printf("%d * %d = %d \n", n, i, n * i);
  }
  return 0;
}
```

```c
//Write a program for finding the average of first N natural numbers using a do
    ...while() loop in C.
#include <stdio.h>
int main() {
    int sum = 0, N, i = 1;
    float avg;
    printf("Enter the value of N : ");
    scanf("%d", &N);
    do {
        sum += i;
        i++;
    } while (i <= N);

    printf("Sum : %d", sum);
    avg = (float)sum / N;
    printf("\nAverage of %d numbers : %0.2f", N, avg);
    return 0;
}
```

What is the Fibonacci series in C?

The Fibonacci sequence is a sequence where the next term is the sum of the previous two terms. The first two terms of the Fibonacci sequence are 0 followed by 1

The Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21

```c
#include <stdio.h>
int main() {
  int i, n;
  int fib1 = 0, fib2 = 1;
  int fib3= fib1 + fib2;
  printf("Enter the number of terms: ");
  scanf("%d", &n);
  printf("Fibonacci Series: %d, %d, ", fib1, fib2);
  for (i = 3; i <= n; ++i) {
    printf("%d, ", fib3);
    fib1 = fib2;
    fib2 = fib3;
    fib3 = fib1 + fib2;
  }
  return 0;
}
```

```
Enter the number of terms: 5
Fibonacci Series: 0, 1, 1, 2, 3,
```

```c
//Check whether the number is Prime or not

#include<stdio.h>
int main()
{
    int n,i,m=0,flag=0;
    printf("Enter the number to check prime:");
    scanf("%d",&n);
    m=n/2;
    for(i=2;i<=m;i++)
    {
        if(n%i==0)
        {
            printf("Number is not prime");
            flag=1;
            break;
        }
    }
    if(flag==0)
        printf("Number is prime");
    return 0;
 }
```

Unconditional Statements/Jumps in Loops

- Unconditional statements are the statements which transfer control or flow of execution unconditionally to another block of statements. These are also called as jump statements.
- When executing the body of loop, sometimes it becomes desirable to skip a part of the loop or to leave the loop as soon as a certain condition occurs.
- *Example, in the case of searching for a particular name in a list containing, say, 100 names, program loop must be terminated as soon as the desired name is found.*

- *There are mainly four unconditional statements namely:*

➢ ***Break statement***

➢ ***Continue statement***

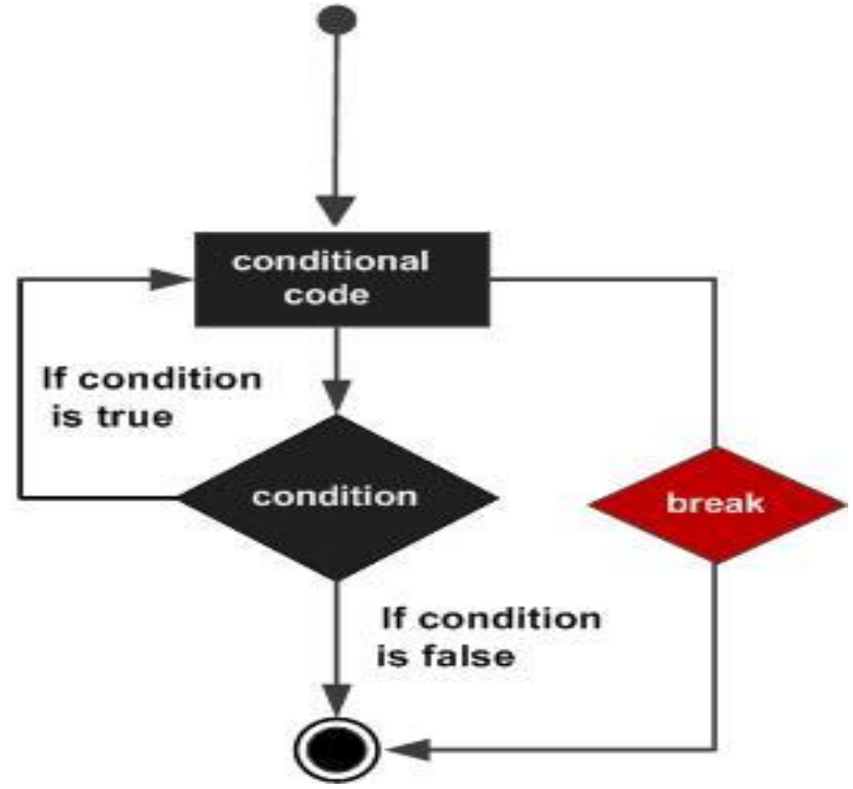
➢ ***Goto statement***

➢ ***Return statement***

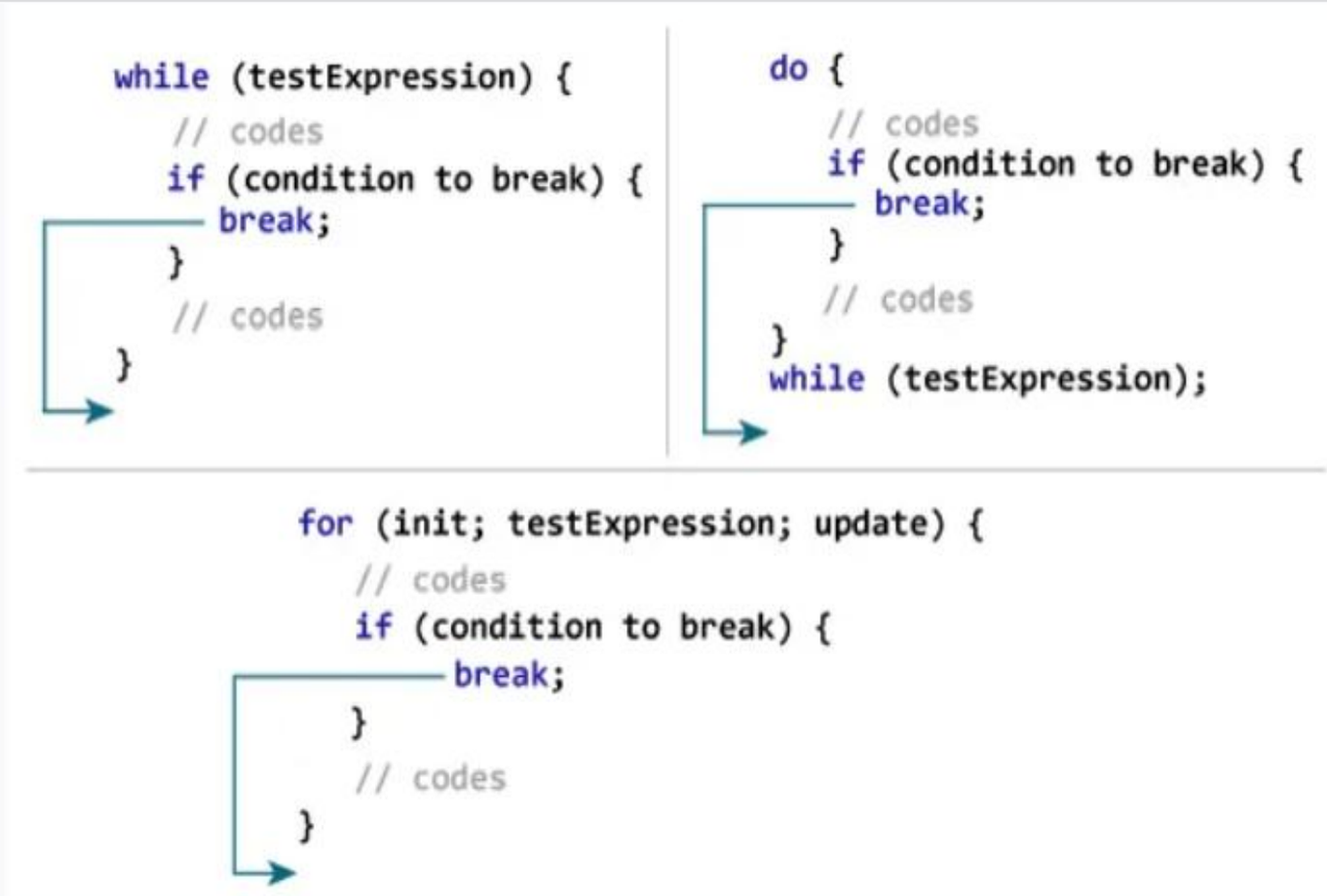
Break statement

- The break is a keyword in C which is used to bring the program control out of the loop.
- The break statement is used inside loops or switch statement.
- The break statement breaks the loop one by one, i.e., in the case of nested loops, it breaks the inner loop first and then proceeds to outer loops.
- The break statement in C can be used in the following two scenarios:
- With switch case
- With loop

- When the **break** statement is encountered *inside the loop*, the loop is immediately exit and program continues with the statement immediately following the loop.
- When the loops are *nested*, the **break** would only exit from the loop containing it. That is **break** will *exit only a single loop*.



How break statement works?



Working of break in C

Examples:

```c
#include<stdio.h>
int main()
{
int num=0;
while(num<=100)
{
   printf("value of variable num is: %d\n", num);
   if (num==2)
  {
      break;
   }
 num++;
}
 printf("Out of while-loop");
}
```

```
value of variable num is: 0
value of variable num is: 1
value of variable num is: 2
Out of while-loop
```

```c
#include<stdio.h>
int main ()
{
    int i;
    for(i = 0; i<10; i++)
    {
        printf("%d ",i);
        if(i == 5)
        break;
    }
    printf("came outside of loop i = %d",i);
   return 0;
}
```

```
0 1 2 3 4 5 came outside of loop i = 5
```

```c
#include<stdio.h>
int main ()
{
    int i = 0;
    while(1)
    {
        printf("%d  ",i);
        i++;
        if(i == 10)
        break;
    }
    printf("came out of while loop");
return 0;
}
```

while(1) acts as an infinite loop that runs continually until a break statement is explicitly issued.

The while(0) loop means that the condition available to us will always be false.

```
0  1  2  3  4  5  6  7  8  9  came out of while loop
```

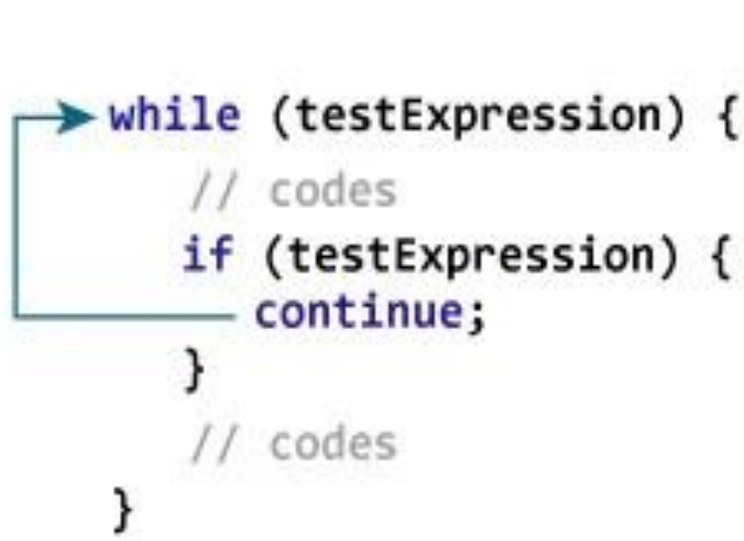
Continue Statement

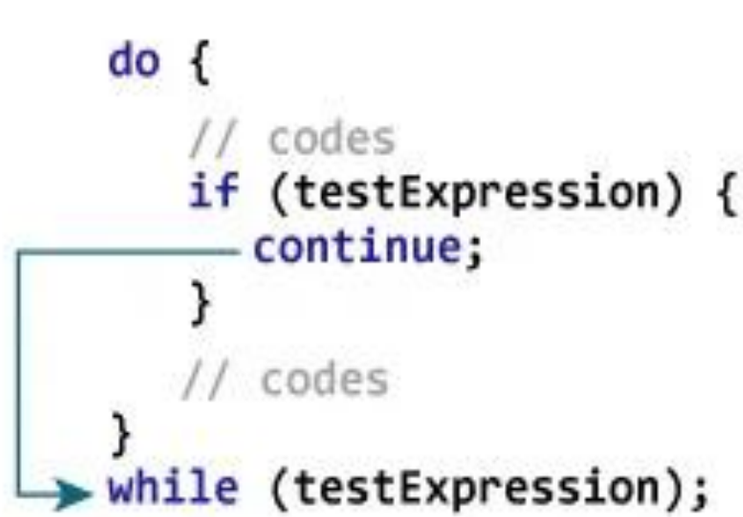
- Continue statement is placed inside a loop to skip the current iteration and proceed to next.
- The continue statement in C programming works somewhat like the break statement.
- Instead of forcing termination, it forces the next iteration of the loop to take place, skipping any code in between.
- For the for loop, continue statement causes the conditional test and increment portions of the loop to execute.
- For the while and do...while loops, continue statement causes the program control to pass to the conditional tests.

Skipping a part of a loop:

- To skip certain statements in the loop and to continue with the next iteration of the loop **continue** statement is used.

```
while (test_condition1)
{
    statement1;
    if (test_condition2)
        continue;   // back to while when test_condition 2 is true
    statement2;
    statement3;
}
```


while (testExpression) {
// codes
if (testExpression) {
continue;
}
// codes
}


do {
// codes
if (testExpression) {
continue;
}
// codes
}
while (testExpression);


for (init; testExpression; update) {
// codes
if (testExpression) {
continue;
}
// codes
}

Example

```c
#include<stdio.h>
int main()
 {
     for (int j=0; j<=8; j++)
     {
          if (j==4)
          {
              continue;
               /* The continue statement is encountered when the
               value of j is equal to 4. */
             }
              /* This print statement would not execute for the loop
                 iteration where j ==4 because in that case this
                 statement would be skipped. */
              printf("%d ", j);
      }
}
```

```
0 1 2 3 5 6 7 8
```

```c
#include<stdio.h>
int main ()
{
   int a = 10;
   do
   {
      if( a == 15)
      {
         a = a + 1;
         continue;
      }
      printf("value of a: %d\n", a);
      a++;
   } while( a < 20 );
   return 0;
}
```

```
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 16
value of a: 17
value of a: 18
value of a: 19
```

goto statement

- The goto statement is an unconditional statement in C which allows the flow of execution of statements to be altered without the use of any conditional expression.
- The goto statement transfers the control from one point to another in a C program. This is also known as Jumping of control or Unconditional Jumping.
- The syntax of goto statement is as follows:

goto label;

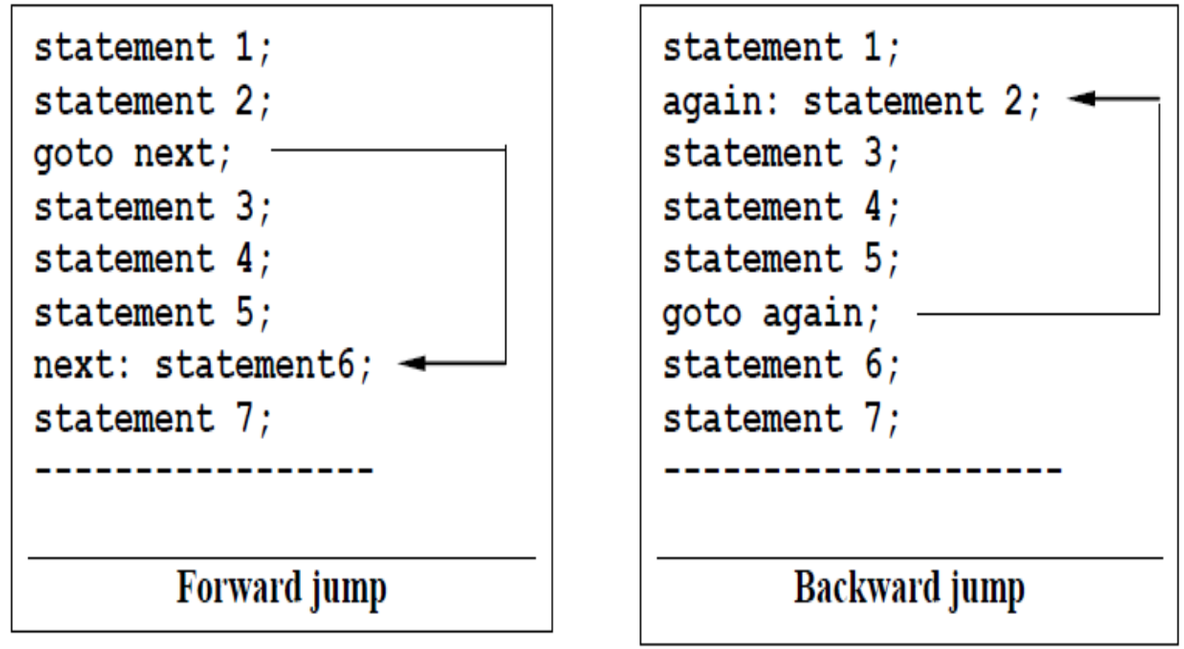
……

label :

- where, goto is a keyword, and
- label is a symbolic constant or an identifier which need not be pre-declared.

- Two types of jumping (branching) is possible by using goto statement:

1. Forward jump: Here the statements immediately following the goto statement will be skipped and control is transferred to the statement beginning with the symbolic constant label.
2. Backward jump: Here the symbolic constant label is placed before the goto statement and the statements between the label and goto statements are repeated resulting in looping.




#include<stdio.h>
#include <conio.h>
int main()
{
printf ("Hello World\n");
goto Label;
printf("How are you?");
printf("Are you Okey?");
Label:
printf("Hope you are fine");
getch();
}
Jumped
Skipped

Output: ??

```
/* Goto Statement in C Programming example */
#include <stdio.h>
#include<stdlib.h>
int main()
{
  int marks;
  printf(" \n Please Enter your Subject Marks \n ");
  scanf("%d", &marks);
  if(marks >= 35)
   {
     goto Pass;
   }
  else
     goto Fail;
  Pass: printf(" \n Congratulation! You made it\n");  exit(0);
  Fail: printf(" \n Better Luck Next Time\n");
 return 0;
}
```

return statement

- Return statement terminates the execution of a function and returns control to the calling function.
- It can also return a value to the calling function.
- Syntax:

return expression;

Here the value of expression is returned.

End of Unit 2

THANK YOU